

**BASIC SQL
ASSIGNMENT 3
WINDOW FUNCTIONS AND CTEs**

SUBMITTED BY: HARSHPREET KAUR
(harshpreet.kaur@kenexai.com)

TASK 1: WINDOW FUNCTIONS - ROW_NUMBER()

--1. Assign unique row number to each patient ordered by registration date

```
SELECT
    patient_id,
    first_name,
    last_name,
    registration_date,
    ROW_NUMBER() OVER(ORDER BY registration_date) AS RowNum
FROM Patient;
```

	patient_id	first_name	last_name	registration_date	RowNum
1	1	Rahul	Shah	2026-06-10	1
2	2	Sneha	Patel	2026-06-10	2
3	3	Arjun	Mehta	2026-06-10	3
4	4	Kavya	Rao	2026-06-10	4
5	5	Rohan	Verma	2026-06-10	5
6	6	Priya	Nair	2026-06-10	6
7	7	Aditya	Gupta	2026-06-10	7
8	8	Pallavi	Joshi	2026-06-10	8
9	9	Nitin	Singh	2026-06-10	9
10	10	Meera	Kapoor	2026-06-10	10
11	11	Aakash	Sharma	2026-06-10	11
12	12	Ritika	Das	2026-06-10	12
13	13	Vivek	Reddy	2026-06-10	13
14	14	Ananya	Iyer	2026-06-10	14
15	15	Deepak	Malhotra	2026-06-10	15
16	16	Pooja	Kulkarni	2026-06-10	16
17	17	Sanjay	Tiwari	2026-06-10	17
18	18	Neelam	Bansal	2026-06-10	18
19	19	Harsh	Chopra	2026-06-10	19
20	20	Divya	Menon	2026-06-10	20

--2. Number doctors within each department ordered by years of service

```
SELECT
    doctor_id,
    first_name,
    last_name,
    dept_id,
    ROW_NUMBER() OVER
    (
        PARTITION BY dept_id
        ORDER BY doctor_id
    ) AS Doctor_Number
FROM Doctor;
```

	doctor_id	first_name	last_name	dept_id	Doctor_Number
1	1	Amit	Sharma	1	1
2	2	Neha	Patel	1	2
3	3	Raj	Verma	2	1
4	4	Priya	Singh	2	2
5	5	Anjali	Gupta	3	1
6	6	Karan	Mehta	3	2
7	7	Vikas	Joshi	4	1
8	8	Ritu	Nair	4	2
9	9	Suresh	Iyer	5	1
10	10	Pooja	Reddy	5	2

--3. Rank appointments for each patient by appointment date

```

SELECT
  patient_id,
  appt_id,
  appointment_date,
  ROW_NUMBER() OVER
  (
    PARTITION BY patient_id
    ORDER BY appointment_date DESC
  ) AS Appointment_Rank
FROM Appointment;

```

	patient_id	appt_id	appointment_date	Appointment_Rank
1	1	21	2026-07-22	1
2	1	1	2025-01-10	2
3	2	22	2026-07-25	1
4	2	2	2025-01-12	2
5	3	23	2026-07-28	1
6	3	3	2025-01-15	2
7	4	24	2026-08-01	1
8	4	4	2025-01-18	2
9	5	25	2026-08-03	1
10	5	5	2025-01-20	2
11	6	26	2025-03-10	1
12	6	6	2025-01-22	2
13	7	27	2025-03-12	1
14	7	7	2025-01-25	2
15	8	28	2025-03-15	1

16	8	8	2025-01-28	2
17	9	29	2025-03-18	1
18	9	9	2025-02-01	2
19	10	30	2025-03-20	1
20	10	10	2025-02-03	2
21	11	11	2025-02-05	1
22	12	12	2025-02-08	1
23	13	13	2025-02-10	1
24	14	14	2025-02-12	1
25	15	15	2025-02-15	1
26	16	16	2026-07-10	1
27	17	17	2026-07-12	1
28	18	18	2026-07-15	1
29	19	19	2026-07-18	1
30	20	20	2026-07-20	1

--4. Number billing records for each patient

```

SELECT
  patient_id,
  bill_id,
  amount,
  ROW_NUMBER() OVER
  (
    PARTITION BY patient_id
    ORDER BY bill_date
  ) AS Bill_Number
FROM Billing;

```

	patient_id	bill_id	amount	Bill_Number
1	1	1	2500.00	1
2	2	2	12000.00	1
3	3	3	1800.00	1
4	4	4	15000.00	1
5	5	5	2200.00	1
6	6	6	1200.00	1
7	7	7	18000.00	1
8	8	8	3500.00	1
9	9	9	5000.00	1
10	10	10	4200.00	1
11	11	11	2500.00	1
12	12	12	3000.00	1
13	13	13	4500.00	1
14	14	14	2000.00	1
15	15	15	8000.00	1
16	16	16	1500.00	1
17	17	17	25000.00	1
18	18	18	900.00	1
19	19	19	50000.00	1
20	20	20	7000.00	1

--5. Create sequential record numbers within each diagnosis history

```
SELECT
  patient_id,
  diagnosis,
  record_date,
  ROW_NUMBER() OVER
  (
    PARTITION BY patient_id
    ORDER BY record_date
  ) AS Diagnosis_Order
FROM Medical_Record;
```

	patient_id	diagnosis	record_date	Diagnosis_Order
1	1	Hypertension	2025-01-10	1
2	2	Heart Arrhythmia	2025-01-12	1
3	3	Migraine	2025-01-15	1
4	4	Epilepsy	2025-01-18	1
5	5	Asthma	2025-01-20	1
6	6	Viral Fever	2025-01-22	1
7	7	Fractured Arm	2025-01-25	1
8	8	Knee Pain	2025-01-28	1
9	9	Food Poisoning	2025-02-01	1
10	10	Chest Infection	2025-02-03	1
11	11	Diabetes	2025-02-05	1
12	12	Anxiety	2025-02-08	1
13	13	Back Pain	2025-02-10	1
14	14	Allergy	2025-02-12	1
15	15	High Cholesterol	2025-02-15	1

TASK 2: RANKING FUNCTIONS - RANK & DENSE_RANK

--1. Top Billing Analysis

SELECT

```
bill_id,  
patient_id,  
amount,  
RANK() OVER(ORDER BY amount DESC) AS Rank_No,  
DENSE_RANK() OVER(ORDER BY amount DESC) AS Dense_Rank_No  
FROM Billing;
```

	bill_id	patient_id	amount	Rank_No	Dense_Rank_No
1	19	19	50000.00	1	1
2	17	17	25000.00	2	2
3	7	7	18000.00	3	3
4	4	4	15000.00	4	4
5	2	2	12000.00	5	5
6	15	15	8000.00	6	6
7	20	20	7000.00	7	7
8	9	9	5000.00	8	8
9	13	13	4500.00	9	9
10	10	10	4200.00	10	10
11	8	8	3500.00	11	11
12	12	12	3000.00	12	12
13	1	1	2500.00	13	13
14	11	11	2500.00	13	13
15	5	5	2200.00	15	14
16	14	14	2000.00	16	15
17	3	3	1800.00	17	16
18	16	16	1500.00	18	17
19	6	6	1200.00	19	18
20	18	18	900.00	20	19

--2. Doctor Performance

SELECT

```
d.doctor_id,  
d.first_name,  
d.last_name,  
dp.dept_name,  
COUNT(a.appt_id) AS Completed_Appointments,  
  
RANK() OVER  
(  
    PARTITION BY dp.dept_name  
    ORDER BY COUNT(a.appt_id) DESC  
) AS Doctor_Rank  
  
FROM Doctor d  
JOIN Department dp  
ON d.dept_id = dp.dept_id
```

```

LEFT JOIN Appointment a
ON d.doctor_id = a.doctor_id
AND a.status='Completed'

```

```

GROUP BY
d.doctor_id,
d.first_name,
d.last_name,
dp.dept_name;

```

	doctor_id	first_name	last_name	dept_name	Completed_Appointments	Doctor_Rank
1	1	Amit	Sharma	Cardiology	2	1
2	2	Neha	Patel	Cardiology	2	1
3	9	Suresh	Iyer	Emergency	1	1
4	10	Pooja	Reddy	Emergency	1	1
5	3	Raj	Verma	Neurology	2	1
6	4	Priya	Singh	Neurology	2	1
7	7	Vikas	Joshi	Orthopedics	1	1
8	8	Ritu	Nair	Orthopedics	1	1
9	5	Anjali	Gupta	Pediatrics	2	1
10	6	Karan	Mehta	Pediatrics	1	2

--3. Patient Visit Frequency

```

SELECT
TOP 10
patient_id,
COUNT(*) AS Visit_Count,

RANK() OVER
(
ORDER BY COUNT(*) DESC
) AS Visit_Rank

FROM Appointment
GROUP BY patient_id;

```

	patient_id	Visit_Count	Visit_Rank
1	1	2	1
2	2	2	1
3	3	2	1
4	4	2	1
5	5	2	1
6	6	2	1
7	7	2	1
8	8	2	1
9	9	2	1
10	10	2	1

--4. Department Revenue Ranking

```
SELECT
  dp.dept_name,
  SUM(b.amount) AS Revenue,

  DENSE_RANK() OVER
  (
    ORDER BY SUM(b.amount) DESC
  ) AS Revenue_Rank

FROM Department dp
JOIN Doctor d
ON dp.dept_id=d.dept_id

JOIN Appointment a
ON d.doctor_id=a.doctor_id

JOIN Billing b
ON a.patient_id=b.patient_id

GROUP BY dp.dept_name;
```

	dept_name	Revenue	Revenue_Rank
1	Emergency	87700.00	1
2	Orthopedics	50800.00	2
3	Neurology	37800.00	3
4	Pediatrics	29700.00	4
5	Cardiology	29200.00	5

--5. Prescription Frequency

```
SELECT
  medication,
  COUNT(*) AS Times_Prescribed,

  RANK() OVER
  (
    ORDER BY COUNT(*) DESC
  ) AS Medication_Rank

FROM Prescription
GROUP BY medication;
```


	medication	Times_Prescribed	Medication_Rank
1	Amlodipine	1	1
2	Antibiotic	1	1
3	Aspirin	1	1
4	Atorvastatin	1	1
5	Azithromycin	1	1
6	Calcium Supplement	1	1
7	Cetirizine	1	1
8	Cough Syrup	1	1
9	Escitalopram	1	1
10	Ibuprofen	1	1
11	Levetiracetam	1	1
12	Losartan	1	1
13	Metformin	1	1
14	Metoprolol	1	1
15	Montelukast	1	1
16	Muscle Relaxant	1	1
17	Naproxen	1	1
18	ORS	1	1
19	Pain Gel	1	1
20	Pain Reliever	1	1
21	Paracetamol	1	1
22	Salbutamol Inhaler	1	1
23	Sumatriptan	1	1
24	Vitamin B Complex	1	1
25	Vitamin D	1	1

TASK 3: LAG & LEAD FUNCTIONS

--1. Appointment Follow-Ups

SELECT

patient_id,
appointment_date,

LAG(appointment_date)

OVER

(
PARTITION BY patient_id
ORDER BY appointment_date
) AS Previous_Appointment,

DATEDIFF

(
DAY,
LAG(appointment_date)
OVER
(
PARTITION BY patient_id
ORDER BY appointment_date
)
)
appointment_date
) AS Days_Between

FROM Appointment;

	patient_id	appointment_date	Previous_Appointment	Days_Between
1	1	2025-01-10	NULL	NULL
2	1	2026-07-22	2025-01-10	558
3	2	2025-01-12	NULL	NULL
4	2	2026-07-25	2025-01-12	559
5	3	2025-01-15	NULL	NULL
6	3	2026-07-28	2025-01-15	559
7	4	2025-01-18	NULL	NULL
8	4	2026-08-01	2025-01-18	560
9	5	2025-01-20	NULL	NULL
10	5	2026-08-03	2025-01-20	560
11	6	2025-01-22	NULL	NULL
12	6	2025-03-10	2025-01-22	47
13	7	2025-01-25	NULL	NULL
14	7	2025-03-12	2025-01-25	46
15	8	2025-01-28	NULL	NULL

16	8	2025-03-15	2025-01-28	46
17	9	2025-02-01	NULL	NULL
18	9	2025-03-18	2025-02-01	45
19	10	2025-02-03	NULL	NULL
20	10	2025-03-20	2025-02-03	45
21	11	2025-02-05	NULL	NULL
22	12	2025-02-08	NULL	NULL
23	13	2025-02-10	NULL	NULL
24	14	2025-02-12	NULL	NULL
25	15	2025-02-15	NULL	NULL
26	16	2026-07-10	NULL	NULL
27	17	2026-07-12	NULL	NULL
28	18	2026-07-15	NULL	NULL
29	19	2026-07-18	NULL	NULL
30	20	2026-07-20	NULL	NULL

--2. Billing Trends

SELECT

patient_id,
bill_date,
amount,

LAG(amount)

OVER

(

PARTITION BY patient_id

ORDER BY bill_date

) AS Previous_Bill,

amount -

LAG(amount)

OVER

(

PARTITION BY patient_id

ORDER BY bill_date

) AS Difference

FROM Billing;

	patient_id	bill_date	amount	Previous_Bill	Difference
1	1	2025-01-10	2500.00	NULL	NULL
2	2	2025-01-12	12000.00	NULL	NULL
3	3	2025-01-15	1800.00	NULL	NULL
4	4	2025-01-18	15000.00	NULL	NULL
5	5	2025-01-20	2200.00	NULL	NULL
6	6	2025-01-22	1200.00	NULL	NULL
7	7	2025-01-25	18000.00	NULL	NULL
8	8	2025-01-28	3500.00	NULL	NULL
9	9	2025-02-01	5000.00	NULL	NULL
10	10	2025-02-03	4200.00	NULL	NULL
11	11	2025-02-05	2500.00	NULL	NULL
12	12	2025-02-08	3000.00	NULL	NULL
13	13	2025-02-10	4500.00	NULL	NULL
14	14	2025-02-12	2000.00	NULL	NULL
15	15	2025-02-15	8000.00	NULL	NULL
16	16	2025-03-01	1500.00	NULL	NULL
17	17	2025-03-05	25000.00	NULL	NULL
18	18	2025-03-08	900.00	NULL	NULL
19	19	2025-03-12	50000.00	NULL	NULL
20	20	2025-03-15	7000.00	NULL	NULL

--3. Doctor Consultation Fee Changes

SELECT

doctor_id,
first_name,
last_name,

LEAD(doctor_id)
OVER(ORDER BY doctor_id) AS Next_Doctor

FROM Doctor;

	doctor_id	first_name	last_name	Next_Doctor
1	1	Amit	Sharma	2
2	2	Neha	Patel	3
3	3	Raj	Verma	4
4	4	Priya	Singh	5
5	5	Anjali	Gupta	6
6	6	Karan	Mehta	7
7	7	Vikas	Joshi	8
8	8	Ritu	Nair	9
9	9	Suresh	Iyer	10
10	10	Pooja	Reddy	NULL

--4. Department Appointment Flow

WITH DailyAppointments AS

```
(
  SELECT
    d.dept_id,
    dp.dept_name,
    appointment_date,
    COUNT(*) AS Daily_Count

  FROM Appointment a
  JOIN Doctor d
    ON a.doctor_id=d.doctor_id
  JOIN Department dp
    ON d.dept_id=dp.dept_id

  GROUP BY
    d.dept_id,
    dp.dept_name,
    appointment_date
)
```

```
SELECT
  dept_name,
  appointment_date,
  Daily_Count,

  LAG(Daily_Count)
  OVER
  (
    PARTITION BY dept_name
    ORDER BY appointment_date
  ) AS Previous_Day_Count
```

FROM DailyAppointments;

	dept_name	appointment_date	Daily_Count	Previous_Day_Count
1	Cardiology	2025-01-10	1	NULL
2	Cardiology	2025-01-12	1	1
3	Cardiology	2025-02-05	1	1
4	Cardiology	2025-02-08	1	1
5	Cardiology	2025-03-18	1	1
6	Cardiology	2025-03-20	1	1
7	Emergency	2025-02-01	1	NULL
8	Emergency	2025-02-03	1	1
9	Emergency	2025-03-12	1	1
10	Emergency	2025-03-15	1	1
11	Emergency	2026-07-18	1	1
12	Emergency	2026-07-20	1	1
13	Neurology	2025-01-15	1	NULL
14	Neurology	2025-01-18	1	1
15	Neurology	2025-02-10	1	1

16	Neurology	2025-02-12	1	1
17	Neurology	2026-07-22	1	1
18	Neurology	2026-07-25	1	1
19	Orthopedics	2025-01-25	1	NULL
20	Orthopedics	2025-01-28	1	1
21	Orthopedics	2025-03-10	1	1
22	Orthopedics	2026-07-12	1	1
23	Orthopedics	2026-07-15	1	1
24	Orthopedics	2026-08-03	1	1
25	Pediatrics	2025-01-20	1	NULL
26	Pediatrics	2025-01-22	1	1
27	Pediatrics	2025-02-15	1	1
28	Pediatrics	2026-07-10	1	1
29	Pediatrics	2026-07-28	1	1
30	Pediatrics	2026-08-01	1	1

--5. Patient Health Progression

SELECT

patient_id,
record_date,
diagnosis,

LAG(diagnosis)

OVER

(
PARTITION BY patient_id
ORDER BY record_date
) AS Previous_Diagnosis

FROM Medical_Record;

	patient_id	record_date	diagnosis	Previous_Diagnosis
1	1	2025-01-10	Hypertension	NULL
2	2	2025-01-12	Heart Arrhythmia	NULL
3	3	2025-01-15	Migraine	NULL
4	4	2025-01-18	Epilepsy	NULL
5	5	2025-01-20	Asthma	NULL
6	6	2025-01-22	Viral Fever	NULL
7	7	2025-01-25	Fractured Arm	NULL
8	8	2025-01-28	Knee Pain	NULL
9	9	2025-02-01	Food Poisoning	NULL
10	10	2025-02-03	Chest Infection	NULL
11	11	2025-02-05	Diabetes	NULL
12	12	2025-02-08	Anxiety	NULL
13	13	2025-02-10	Back Pain	NULL
14	14	2025-02-12	Allergy	NULL
15	15	2025-02-15	High Cholesterol	NULL

TASK 4: AGGREGATE WINDOW FUNCTIONS

--1. Running Total Revenue

```
SELECT
    bill_date,
    SUM(amount) AS Daily_Revenue,

    SUM(SUM(amount))
    OVER
    (
        ORDER BY bill_date
    ) AS Running_Total

FROM Billing
GROUP BY bill_date;
```

	bill_date	Daily_Revenue	Running_Total
1	2025-01-10	2500.00	2500.00
2	2025-01-12	12000.00	14500.00
3	2025-01-15	1800.00	16300.00
4	2025-01-18	15000.00	31300.00
5	2025-01-20	2200.00	33500.00
6	2025-01-22	1200.00	34700.00
7	2025-01-25	18000.00	52700.00
8	2025-01-28	3500.00	56200.00
9	2025-02-01	5000.00	61200.00
10	2025-02-03	4200.00	65400.00
11	2025-02-05	2500.00	67900.00
12	2025-02-08	3000.00	70900.00
13	2025-02-10	4500.00	75400.00
14	2025-02-12	2000.00	77400.00
15	2025-02-15	8000.00	85400.00
16	2025-03-01	1500.00	86900.00
17	2025-03-05	25000.00	111900.00
18	2025-03-08	900.00	112800.00
19	2025-03-12	50000.00	162800.00
20	2025-03-15	7000.00	169800.00

--2. Department Comparison

```
SELECT
    d.doctor_id,
    d.first_name,
    d.last_name,

    AVG(d.doctor_id * 1000.0)
    OVER(PARTITION BY dept_id) AS Dept_Avg_Fee

FROM Doctor d;
```

	doctor_id	first_name	last_name	Dept_Avg_Fee
1	1	Amit	Sharma	1500.000000
2	2	Neha	Patel	1500.000000
3	3	Raj	Verma	3500.000000
4	4	Priya	Singh	3500.000000
5	5	Anjali	Gupta	5500.000000
6	6	Karan	Mehta	5500.000000
7	7	Vikas	Joshi	7500.000000
8	8	Ritu	Nair	7500.000000
9	9	Suresh	Iyer	9500.000000
10	10	Pooja	Reddy	9500.000000

--3. Patient Billing Summary

```

SELECT
  patient_id,
  bill_id,
  amount,
  SUM(amount)
  OVER
  (
    PARTITION BY patient_id
    ORDER BY bill_date
  ) AS Running_Total,
  ROUND
  (
    amount * 100.0 /
    SUM(amount)
    OVER(PARTITION BY patient_id),
    2
  ) AS Percent_Contribution
FROM Billing;

```

	patient_id	bill_id	amount	Running_Total	Percent_Contribution
1	1	1	2500.00	2500.00	100.000000000000000000000000000000
2	2	2	12000.00	12000.00	100.000000000000000000000000000000
3	3	3	1800.00	1800.00	100.000000000000000000000000000000
4	4	4	15000.00	15000.00	100.000000000000000000000000000000
5	5	5	2200.00	2200.00	100.000000000000000000000000000000
6	6	6	1200.00	1200.00	100.000000000000000000000000000000
7	7	7	18000.00	18000.00	100.000000000000000000000000000000
8	8	8	3500.00	3500.00	100.000000000000000000000000000000
9	9	9	5000.00	5000.00	100.000000000000000000000000000000
10	10	10	4200.00	4200.00	100.000000000000000000000000000000
11	11	11	2500.00	2500.00	100.000000000000000000000000000000
12	12	12	3000.00	3000.00	100.000000000000000000000000000000
13	13	13	4500.00	4500.00	100.000000000000000000000000000000
14	14	14	2000.00	2000.00	100.000000000000000000000000000000
15	15	15	8000.00	8000.00	100.000000000000000000000000000000
16	16	16	1500.00	1500.00	100.000000000000000000000000000000
17	17	17	25000.00	25000.00	100.000000000000000000000000000000
18	18	18	900.00	900.00	100.000000000000000000000000000000
19	19	19	50000.00	50000.00	100.000000000000000000000000000000
20	20	20	7000.00	7000.00	100.000000000000000000000000000000

--4. Monthly Appointment Trends

```
SELECT
  YEAR(appointment_date) AS Yr,
  MONTH(appointment_date) AS Mn,

  COUNT(*) AS Monthly_Appointments,

  SUM(COUNT(*))
  OVER
  (
    ORDER BY
      YEAR(appointment_date),
      MONTH(appointment_date)
  ) AS Cumulative_Appointments

FROM Appointment

GROUP BY
  YEAR(appointment_date),
  MONTH(appointment_date);
```

	Yr	Mn	Monthly_Appointments	Cumulative_Appointments
1	2025	1	8	8
2	2025	2	7	15
3	2025	3	5	20
4	2026	7	8	28
5	2026	8	2	30

--5. Doctor Workload Analysis

```
SELECT
  d.doctor_id,
  d.first_name,
  d.last_name,

  COUNT(a.appt_id) AS Doctor_Appointments,

  SUM(COUNT(a.appt_id))
  OVER
  (
    PARTITION BY d.dept_id
  ) AS Department_Total,

  ROUND
  (
    COUNT(a.appt_id)*100.0/
    SUM(COUNT(a.appt_id))
    OVER(PARTITION BY d.dept_id),
    2
  ) AS Workload_Percentage
```

```
FROM Doctor d
LEFT JOIN Appointment a
ON d.doctor_id=a.doctor_id
```

```
GROUP BY
d.doctor_id,
d.first_name,
d.last_name,
d.dept_id;
```

	doctor_id	first_name	last_name	Doctor_Appointments	Department_Total	Workload_Percentage
1	1	Amit	Sharma	3	6	50.000000000000
2	2	Neha	Patel	3	6	50.000000000000
3	3	Raj	Verma	3	6	50.000000000000
4	4	Priya	Singh	3	6	50.000000000000
5	5	Anjali	Gupta	3	6	50.000000000000
6	6	Karan	Mehta	3	6	50.000000000000
7	7	Vikas	Joshi	3	6	50.000000000000
8	8	Ritu	Nair	3	6	50.000000000000
9	9	Suresh	Iyer	3	6	50.000000000000
10	10	Pooja	Reddy	3	6	50.000000000000

TASK 5: BASIC CTEs

--1. High value Patients

WITH PatientSpending AS

```
(
  SELECT
    p.patient_id,
    p.first_name,
    p.last_name,
    SUM(b.amount) AS Total_Spending
  FROM Patient p
  JOIN Billing b
    ON p.patient_id = b.patient_id
  GROUP BY
    p.patient_id,
    p.first_name,
    p.last_name
)
```

```
SELECT *
FROM PatientSpending
WHERE Total_Spending > 15000;
```

	patient_id	first_name	last_name	Total_Spending
1	7	Aditya	Gupta	18000.00
2	17	Sanjay	Tiwari	25000.00
3	19	Harsh	Chopra	50000.00

--2. Busy Doctors

WITH DoctorAppointments AS

```
(
  SELECT
    d.doctor_id,
    d.first_name,
    d.last_name,
    COUNT(a.appt_id) AS Appointment_Count
  FROM Doctor d
  LEFT JOIN Appointment a
    ON d.doctor_id = a.doctor_id
  GROUP BY
    d.doctor_id,
    d.first_name,
    d.last_name
)
```

```
SELECT *
FROM DoctorAppointments
WHERE Appointment_Count > 3;
```

	doctor_id	first_name	last_name	Appointment_Count
--	-----------	------------	-----------	-------------------

--3. Department Averages

```
WITH DepartmentRevenue AS
(
    SELECT
        dp.dept_id,
        dp.dept_name,
        AVG(b.amount) AS Avg_Bill
    FROM Department dp
    JOIN Doctor d
        ON dp.dept_id = d.dept_id
    JOIN Appointment a
        ON d.doctor_id = a.doctor_id
    JOIN Billing b
        ON a.patient_id = b.patient_id
    GROUP BY
        dp.dept_id,
        dp.dept_name
)
```

```
SELECT *
FROM DepartmentRevenue
WHERE Avg_Bill >
(
    SELECT AVG(Avg_Bill)
    FROM DepartmentRevenue
);
```

	dept_id	dept_name	Avg_Bill
1	4	Orthopedics	8466.666666
2	5	Emergency	14616.666666

--4. Recent Patients

```
WITH RecentPatients AS
(
    SELECT
        patient_id,
        first_name,
        last_name,
        registration_date
    FROM Patient
    WHERE registration_date >= DATEADD(DAY,-180,GETDATE())
)
```

```
SELECT
    rp.patient_id,
    rp.first_name,
    rp.last_name,
    rp.registration_date,
    COUNT(a.appt_id) AS Appointment_Count
FROM RecentPatients rp
```

```

LEFT JOIN Appointment a
  ON rp.patient_id = a.patient_id
GROUP BY
  rp.patient_id,
  rp.first_name,
  rp.last_name,
  rp.registration_date;

```

	patient_id	first_name	last_name	registration_date	Appointment_Count
1	1	Rahul	Shah	2026-06-10	2
2	2	Sneha	Patel	2026-06-10	2
3	3	Arjun	Mehta	2026-06-10	2
4	4	Kavya	Rao	2026-06-10	2
5	5	Rohan	Verma	2026-06-10	2
6	6	Priya	Nair	2026-06-10	2
7	7	Aditya	Gupta	2026-06-10	2
8	8	Pallavi	Joshi	2026-06-10	2
9	9	Nitin	Singh	2026-06-10	2
10	10	Meera	Kapoor	2026-06-10	2
11	11	Aakash	Sharma	2026-06-10	1
12	12	Ritika	Das	2026-06-10	1
13	13	Vivek	Reddy	2026-06-10	1
14	14	Ananya	Iyer	2026-06-10	1
15	15	Deepak	Malhotra	2026-06-10	1
16	16	Pooja	Kulkarni	2026-06-10	1
17	17	Sanjay	Tiwari	2026-06-10	1
18	18	Neelam	Bansal	2026-06-10	1
19	19	Harsh	Chopra	2026-06-10	1
20	20	Divya	Menon	2026-06-10	1

--5. Unpaid Bills Summary

```

WITH OutstandingBills AS

```

```

(
  SELECT
    patient_id,
    amount
  FROM Billing
  WHERE payment_status IN ('Pending', 'Cancelled')
)

```

```

SELECT
  p.patient_id,
  p.first_name,
  p.last_name,
  SUM(ob.amount) AS Outstanding_Amount
FROM Patient p
JOIN OutstandingBills ob
  ON p.patient_id = ob.patient_id
GROUP BY
  p.patient_id,
  p.first_name,

```

p.last_name
ORDER BY Outstanding_Amount DESC;

	patient_id	first_name	last_name	Outstanding_Amount
1	19	Harsh	Chopra	50000.00
2	9	Nitin	Singh	5000.00
3	12	Ritika	Das	3000.00
4	14	Ananya	Iyer	2000.00
5	16	Pooja	Kulkarni	1500.00
6	6	Priya	Nair	1200.00

TASK 6: MULTIPLE CTEs

--1. Three-Tier Patient Analysis

WITH AppointmentStats AS

```
(
  SELECT
    patient_id,
    COUNT(appt_id) AS Total_Appointments
  FROM Appointment
  GROUP BY patient_id
),
```

PatientSpending AS

```
(
  SELECT
    patient_id,
    SUM(amount) AS Total_Spending
  FROM Billing
  GROUP BY patient_id
)
```

SELECT

```
p.patient_id,
p.first_name,
p.last_name,
a.Total_Appointments,
s.Total_Spending,
```

CASE

```
  WHEN a.Total_Appointments > 5
        AND s.Total_Spending > 20000
  THEN 'VIP'
```

```
  WHEN a.Total_Appointments BETWEEN 3 AND 5
        OR s.Total_Spending BETWEEN 10000 AND 20000
  THEN 'Regular'
```

```
  ELSE 'Occasional'
END AS Patient_Category
```

FROM Patient p

LEFT JOIN AppointmentStats a

```
  ON p.patient_id = a.patient_id
```

LEFT JOIN PatientSpending s

```
  ON p.patient_id = s.patient_id
```

ORDER BY s.Total_Spending DESC;

	patient_id	first_name	last_name	Total_Appointments	Total_Spending	Patient_Category
1	19	Harsh	Chopra	1	50000.00	Occasional
2	17	Sanjay	Tiwari	1	25000.00	Occasional
3	7	Aditya	Gupta	2	18000.00	Regular
4	4	Kavya	Rao	2	15000.00	Regular
5	2	Sneha	Patel	2	12000.00	Regular
6	15	Deepak	Malhotra	1	8000.00	Occasional
7	20	Divya	Menon	1	7000.00	Occasional
8	9	Nitin	Singh	2	5000.00	Occasional
9	13	Vivek	Reddy	1	4500.00	Occasional
10	10	Meera	Kapoor	2	4200.00	Occasional
11	8	Pallavi	Joshi	2	3500.00	Occasional
12	12	Ritika	Das	1	3000.00	Occasional
13	1	Rahul	Shah	2	2500.00	Occasional
14	11	Aakash	Sharma	1	2500.00	Occasional
15	5	Rohan	Verma	2	2200.00	Occasional
16	14	Ananya	Iyer	1	2000.00	Occasional
17	3	Arjun	Mehta	2	1800.00	Occasional
18	16	Pooja	Kulkarni	1	1500.00	Occasional
19	6	Priya	Nair	2	1200.00	Occasional
20	18	Neelam	Bansal	1	900.00	Occasional

--2. Department Performance Dashboard

WITH DoctorCount AS

```
(
  SELECT
    dept_id,
    COUNT(*) AS Total_Doctors
  FROM Doctor
  GROUP BY dept_id
),
```

AppointmentCount AS

```
(
  SELECT
    d.dept_id,
    COUNT(a.appt_id) AS Total_Appointments
  FROM Doctor d
  LEFT JOIN Appointment a
    ON d.doctor_id = a.doctor_id
  GROUP BY d.dept_id
),
```

DepartmentRevenue AS

```
(
  SELECT
    d.dept_id,
    SUM(b.amount) AS Total_Revenue
  FROM Doctor d
  JOIN Appointment a
    ON d.doctor_id = a.doctor_id
  JOIN Billing b
    ON a.patient_id = b.patient_id
  GROUP BY d.dept_id
)
```



```

SELECT
    dp.dept_name,
    dc.Total_Doctors,
    ac.Total_Appointments,
    dr.Total_Revenue
FROM Department dp
LEFT JOIN DoctorCount dc
    ON dp.dept_id = dc.dept_id
LEFT JOIN AppointmentCount ac
    ON dp.dept_id = ac.dept_id
LEFT JOIN DepartmentRevenue dr
    ON dp.dept_id = dr.dept_id
ORDER BY dr.Total_Revenue DESC;

```

	dept_name	Total_Doctors	Total_Appointments	Total_Revenue
1	Emergency	2	6	87700.00
2	Orthopedics	2	6	50800.00
3	Neurology	2	6	37800.00
4	Pediatrics	2	6	29700.00
5	Cardiology	2	6	29200.00

--3. Billing vs Appointment Analysis

```

WITH AvgBill AS
(
    SELECT
        AVG(amount) AS Average_Bill
    FROM Billing
),

```

```

AppointmentsWithoutBills AS
(
    SELECT
        a.appt_id,
        a.patient_id,
        a.appointment_date
    FROM Appointment a
    LEFT JOIN Billing b
        ON a.patient_id = b.patient_id
    WHERE b.bill_id IS NULL
)

```

```

SELECT
    COUNT(*) AS Unbilled_Appointments,
    ab.Average_Bill,
    COUNT(*) * ab.Average_Bill AS Potential_Revenue_Loss
FROM AppointmentsWithoutBills awb
CROSS JOIN AvgBill ab
GROUP BY ab.Average_Bill;

```

Unbilled_Appointments	Average_Bill	Potential_Revenue_Loss
-----------------------	--------------	------------------------

--4. Top Performing Doctors

WITH CompletedAppointments AS

```
(
  SELECT
    doctor_id,
    COUNT(*) AS Completed_Count
  FROM Appointment
  WHERE status = 'Completed'
  GROUP BY doctor_id
),
```

DoctorRevenue AS

```
(
  SELECT
    a.doctor_id,
    SUM(b.amount) AS Revenue_Generated
  FROM Appointment a
  JOIN Billing b
    ON a.patient_id = b.patient_id
  GROUP BY a.doctor_id
),
```

CancellationRate AS

```
(
  SELECT
    doctor_id,

    CAST(
      SUM(
        CASE
          WHEN status = 'Cancelled'
          THEN 1
          ELSE 0
        END
      ) * 100.0
      / COUNT(*)
      AS DECIMAL(5,2)
    ) AS Cancellation_Rate

  FROM Appointment
  GROUP BY doctor_id
)
```

SELECT

```
d.doctor_id,
d.first_name,
d.last_name,

ISNULL(ca.Completed_Count,0) AS Completed_Appointments,
ISNULL(dr.Revenue_Generated,0) AS Revenue_Generated,
ISNULL(cr.Cancellation_Rate,0) AS Cancellation_Rate,
```

```
(
```

```

        ISNULL(ca.Completed_Count,0) * 10
        +
        ISNULL(dr.Revenue_Generated,0) / 1000.0
        -
        ISNULL(cr.Cancellation_Rate,0)
    ) AS Composite_Score

FROM Doctor d

LEFT JOIN CompletedAppointments ca
    ON d.doctor_id = ca.doctor_id

LEFT JOIN DoctorRevenue dr
    ON d.doctor_id = dr.doctor_id

LEFT JOIN CancellationRate cr
    ON d.doctor_id = cr.doctor_id

ORDER BY Composite_Score DESC;

```

	doctor_id	first_name	last_name	Completed_Appointments	Revenue_Generated	Cancellation_Rate	Composite_Score
1	7	Vikas	Joshi	1	45200.00	0.00	55.200000
2	9	Suresh	Iyer	1	73000.00	33.33	49.670000
3	4	Priya	Singh	2	29000.00	0.00	49.000000
4	5	Anjali	Gupta	2	12000.00	0.00	32.000000
5	3	Raj	Verma	2	8800.00	0.00	28.800000
6	6	Karan	Mehta	1	17700.00	0.00	27.700000
7	2	Neha	Patel	2	19200.00	33.33	5.870000
8	1	Amit	Sharma	2	10000.00	33.33	-3.330000
9	10	Pooja	Reddy	1	14700.00	33.33	-8.630000
10	8	Ritu	Nair	1	5600.00	33.33	-17.730000

TASK 7: CTEs with WINDOW FUNCTIONS

--1. Top 3 Doctors per Department

WITH DoctorRevenue AS

```
(
  SELECT
    d.doctor_id,
    d.first_name,
    d.last_name,
    dp.dept_name,
    SUM(b.amount) AS Revenue

  FROM Doctor d

  JOIN Department dp
    ON d.dept_id = dp.dept_id

  JOIN Appointment a
    ON d.doctor_id = a.doctor_id

  JOIN Billing b
    ON a.patient_id = b.patient_id

  GROUP BY
    d.doctor_id,
    d.first_name,
    d.last_name,
    dp.dept_name
),
```

RankedDoctors AS

```
(
  SELECT *,
    RANK() OVER
      (
        PARTITION BY dept_name
        ORDER BY Revenue DESC
      ) AS DoctorRank
  FROM DoctorRevenue
)
```

```
SELECT *
FROM RankedDoctors
WHERE DoctorRank <= 3
ORDER BY dept_name, DoctorRank;
```

	doctor_id	first_name	last_name	dept_name	Revenue	DoctorRank
1	2	Neha	Patel	Cardiology	19200.00	1
2	1	Amit	Sharma	Cardiology	10000.00	2
3	9	Suresh	Iyer	Emergency	73000.00	1
4	10	Pooja	Reddy	Emergency	14700.00	2
5	4	Priya	Singh	Neurology	29000.00	1
6	3	Raj	Verma	Neurology	8800.00	2
7	7	Vikas	Joshi	Orthopedics	45200.00	1
8	8	Ritu	Nair	Orthopedics	5600.00	2
9	6	Karan	Mehta	Pediatrics	17700.00	1
10	5	Anjali	Gupta	Pediatrics	12000.00	2

--2. Patient Spending Percentile

WITH PatientSpending AS

```
(
  SELECT
    p.patient_id,
    p.first_name,
    p.last_name,
    SUM(b.amount) AS TotalSpending

  FROM Patient p
  JOIN Billing b
    ON p.patient_id = b.patient_id

  GROUP BY
    p.patient_id,
    p.first_name,
    p.last_name
)
```

```
SELECT
  patient_id,
  first_name,
  last_name,
  TotalSpending,

  NTILE(4) OVER
  (
    ORDER BY TotalSpending DESC
  ) AS SpendingQuartile

FROM PatientSpending;
```

	patient_id	first_name	last_name	TotalSpending	SpendingQuartile
1	19	Harsh	Chopra	50000.00	1
2	17	Sanjay	Tiwari	25000.00	1
3	7	Aditya	Gupta	18000.00	1
4	4	Kavya	Rao	15000.00	1
5	2	Sneha	Patel	12000.00	1
6	15	Deepak	Malhotra	8000.00	2
7	20	Divya	Menon	7000.00	2
8	9	Nitin	Singh	5000.00	2
9	13	Vivek	Reddy	4500.00	2
10	10	Meera	Kapoor	4200.00	2
11	8	Pallavi	Joshi	3500.00	3
12	12	Ritika	Das	3000.00	3
13	1	Rahul	Shah	2500.00	3
14	11	Aakash	Sharma	2500.00	3
15	5	Rohan	Verma	2200.00	3
16	14	Ananya	Iyer	2000.00	4
17	3	Arjun	Mehta	1800.00	4
18	16	Pooja	Kulkarni	1500.00	4
19	6	Priya	Nair	1200.00	4
20	18	Neelam	Bansal	900.00	4

--3. Monthly Revenue Growth

WITH MonthlyRevenue AS

```
(
  SELECT
    YEAR(bill_date) AS RevenueYear,
    MONTH(bill_date) AS RevenueMonth,
    SUM(amount) AS Revenue

  FROM Billing

  GROUP BY
    YEAR(bill_date),
    MONTH(bill_date)
)
```

```
SELECT
  RevenueYear,
  RevenueMonth,
  Revenue,

  LAG(Revenue)
  OVER
  (
    ORDER BY RevenueYear, RevenueMonth
  ) AS PreviousMonthRevenue,

  ROUND
  (
```

```

(
    Revenue -
    LAG(Revenue)
    OVER
    (
        ORDER BY RevenueYear, RevenueMonth
    )
) * 100.0

/

NULLIF
(
    LAG(Revenue)
    OVER
    (
        ORDER BY RevenueYear, RevenueMonth
    ),
    0
),
2
) AS GrowthPercentage

FROM MonthlyRevenue;

```

	RevenueYear	RevenueMonth	Revenue	PreviousMonthRevenue	GrowthPercentage
1	2025	1	56200.00	NULL	NULL
2	2025	2	29200.00	56200.00	-48.040000
3	2025	3	84400.00	29200.00	189.040000

--4. Cumulative Patient Registration

```

WITH DailyRegistrations AS
(
    SELECT
        registration_date,
        COUNT(*) AS DailyCount

    FROM Patient

    GROUP BY registration_date
)

SELECT
    registration_date,
    DailyCount,

    SUM(DailyCount)
    OVER
    (
        ORDER BY registration_date
    ) AS CumulativePatients

```

FROM DailyRegistrations;

	registration_date	DailyCount	CumulativePatients
1	2026-06-10	20	20

--5. Department Market Share

WITH DepartmentRevenue AS

```
(
  SELECT
    dp.dept_name,
    SUM(b.amount) AS DepartmentRevenue

  FROM Department dp

  JOIN Doctor d
    ON dp.dept_id = d.dept_id

  JOIN Appointment a
    ON d.doctor_id = a.doctor_id

  JOIN Billing b
    ON a.patient_id = b.patient_id

  GROUP BY dp.dept_name
)
```

```
SELECT
  dept_name,
  DepartmentRevenue,

  ROUND
  (
    DepartmentRevenue * 100.0
    /
    SUM(DepartmentRevenue)
    OVER (),
    2
  ) AS MarketSharePercentage
```

FROM DepartmentRevenue

ORDER BY DepartmentRevenue DESC;

	dept_name	DepartmentRevenue	MarketSharePercentage
1	Emergency	87700.00	37.290000
2	Orthopedics	50800.00	21.600000
3	Neurology	37800.00	16.070000
4	Pediatrics	29700.00	12.630000
5	Cardiology	29200.00	12.410000

TASK 8: RECURSIVE CTEs

--1. Generate Next 30 Days Hospital Calendar

```
WITH HospitalCalendar AS
(
    SELECT
        CAST(GETDATE() AS DATE) AS CalendarDate
    UNION ALL
    SELECT
        DATEADD(DAY, 1, CalendarDate)
    FROM HospitalCalendar
    WHERE CalendarDate <
        DATEADD(DAY, 29, CAST(GETDATE() AS DATE))
)
SELECT *
FROM HospitalCalendar
OPTION (MAXRECURSION 30);
```

	CalendarDate		
1	2026-06-12	16	2026-06-27
2	2026-06-13	17	2026-06-28
3	2026-06-14	18	2026-06-29
4	2026-06-15	19	2026-06-30
5	2026-06-16	20	2026-07-01
6	2026-06-17	21	2026-07-02
7	2026-06-18	22	2026-07-03
8	2026-06-19	23	2026-07-04
9	2026-06-20	24	2026-07-05
10	2026-06-21	25	2026-07-06
11	2026-06-22	26	2026-07-07
12	2026-06-23	27	2026-07-08
13	2026-06-24	28	2026-07-09
14	2026-06-25	29	2026-07-10
15	2026-06-26	30	2026-07-11

--2. Appointment Schedule Timeline

```
WITH DateSeries AS
(
    SELECT CAST('2025-01-10' AS DATE) AS ScheduleDate

    UNION ALL

    SELECT DATEADD(DAY, 1, ScheduleDate)
    FROM DateSeries
    WHERE ScheduleDate < '2026-08-03'
)
```

```

SELECT
    ds.ScheduleDate,
    COUNT(a.appt_id) AS TotalAppointments
FROM DateSeries ds
LEFT JOIN Appointment a
    ON ds.ScheduleDate = a.appointment_date
GROUP BY ds.ScheduleDate
ORDER BY ds.ScheduleDate
OPTION (MAXRECURSION 1000);

```

	ScheduleDate	TotalAppointments
1	2025-01-10	1
2	2025-01-11	0
3	2025-01-12	1
4	2025-01-13	0
5	2025-01-14	0
6	2025-01-15	1
7	2025-01-16	0
8	2025-01-17	0
9	2025-01-18	1
10	2025-01-19	0
11	2025-01-20	1
12	2025-01-21	0
13	2025-01-22	1

--3. Patient Visit Sequence

```

WITH VisitNumbers AS
(
    SELECT 1 AS VisitNo

    UNION ALL

    SELECT VisitNo + 1
    FROM VisitNumbers
    WHERE VisitNo < 10
)
SELECT *
FROM VisitNumbers
OPTION (MAXRECURSION 10);

```

	VisitNo
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10

--4. Department Hierarchy Report

```
WITH DepartmentHierarchy AS
(
    SELECT
        CAST('Hospital' AS VARCHAR(100)) AS NodeName,
        CAST(NULL AS VARCHAR(100)) AS ParentNode,
        0 AS LevelNo

    UNION ALL

    SELECT
        dept_name,
        'Hospital',
        1
    FROM Department
)

SELECT *
FROM DepartmentHierarchy
ORDER BY LevelNo, NodeName;
```

	NodeName	ParentNode	LevelNo
1	Hospital	NULL	0
2	Cardiology	Hospital	1
3	Emergency	Hospital	1
4	Neurology	Hospital	1
5	Orthopedics	Hospital	1
6	Pediatrics	Hospital	1

--5. Doctor Reporting Structure

```
WITH DoctorHierarchy AS
(
    SELECT
        dp.dept_name,
        hd.doctor_id AS HeadDoctorID,
        hd.first_name + ' ' + hd.last_name AS HeadDoctor,
        d.doctor_id,
        d.first_name + ' ' + d.last_name AS DoctorName
    FROM Department dp

    JOIN Doctor hd
        ON dp.head_doctor_id = hd.doctor_id

    JOIN Doctor d
        ON dp.dept_id = d.dept_id
)

SELECT *
FROM DoctorHierarchy
```

ORDER BY dept_name, doctor_id;

	dept_name	HeadDoctorID	HeadDoctor	doctor_id	DoctorName
1	Cardiology	1	Amit Sharma	1	Amit Sharma
2	Cardiology	1	Amit Sharma	2	Neha Patel
3	Emergency	9	Suresh Iyer	9	Suresh Iyer
4	Emergency	9	Suresh Iyer	10	Pooja Reddy
5	Neurology	3	Raj Verma	3	Raj Verma
6	Neurology	3	Raj Verma	4	Priya Singh
7	Orthopedics	7	Vikas Joshi	7	Vikas Joshi
8	Orthopedics	7	Vikas Joshi	8	Ritu Nair
9	Pediatrics	5	Anjali Gupta	5	Anjali Gupta
10	Pediatrics	5	Anjali Gupta	6	Karan Mehta

TASK 9: CTE vs SUBQUERY COMPARISON

1. SIMPLE FILTERING

VERSION A: Subquery

```
SELECT
  p.patient_id,
  p.first_name,
  p.last_name,
  SUM(b.amount) AS TotalSpending
FROM Patient p
JOIN Billing b
  ON p.patient_id = b.patient_id
GROUP BY
  p.patient_id,
  p.first_name,
  p.last_name
HAVING SUM(b.amount) >
(
  SELECT AVG(TotalBill)
  FROM
    (
      SELECT SUM(amount) AS TotalBill
      FROM Billing
      GROUP BY patient_id
    ) AS PatientTotals
);
```

	patient_id	first_name	last_name	TotalSpending
1	2	Sneha	Patel	12000.00
2	4	Kavya	Rao	15000.00
3	7	Aditya	Gupta	18000.00
4	17	Sanjay	Tiwari	25000.00
5	19	Harsh	Chopra	50000.00

EXPLANATION: The query first calculates the total amount spent by each patient using `SUM(amount)` and `GROUP BY patient_id`. Inside the `HAVING` clause, a subquery calculates the average spending across all patients. The outer query then compares each patient's total spending against this average and returns only those patients who have spent more than the average. This approach works well for simple calculations where the result of the subquery is needed only once.

VERSION B: CTE

```
WITH PatientTotals AS
(
```

```

SELECT
    patient_id,
    SUM(amount) AS TotalBill
FROM Billing
GROUP BY patient_id
)

SELECT
    p.patient_id,
    p.first_name,
    p.last_name,
    pt.TotalBill
FROM Patient p
JOIN PatientTotals pt
    ON p.patient_id = pt.patient_id
WHERE pt.TotalBill >
(
    SELECT AVG(TotalBill)
    FROM PatientTotals
);

```

	patient_id	first_name	last_name	TotalBill
1	2	Sneha	Patel	12000.00
2	4	Kavya	Rao	15000.00
3	7	Aditya	Gupta	18000.00
4	17	Sanjay	Tiwari	25000.00
5	19	Harsh	Chopra	50000.00

EXPLANATION: In this version, a CTE named **PatientTotals** is created to calculate each patient's total spending. Once the spending information is stored in the CTE, it can be reused in the main query. The main query compares each patient's spending with the average spending calculated from the same CTE. Using a CTE makes the query easier to understand because the calculation step and filtering step are clearly separated.

Which is Better?

The CTE version is better because it improves readability and makes the query easier to maintain. If additional calculations or filters need to be added later, they can be incorporated into the CTE without making the query difficult to understand,

2. MULTIPLE REFERENCES

VERSION A: Subquery

--Version A: Subquery

```

SELECT
    d.doctor_id,
    d.first_name,

```

```

d.last_name,

COUNT(a.appt_id) AS AppointmentCount,
SUM(b.amount) AS Revenue

FROM Doctor d

JOIN Appointment a
  ON d.doctor_id = a.doctor_id

JOIN Billing b
  ON a.patient_id = b.patient_id

GROUP BY
  d.doctor_id,
  d.first_name,
  d.last_name

HAVING

COUNT(a.appt_id) >
(
  SELECT AVG(AppCount)
  FROM
  (
    SELECT COUNT(*) AS AppCount
    FROM Appointment
    GROUP BY doctor_id
  ) X
)

AND

SUM(b.amount) >
(
  SELECT AVG(Revenue)
  FROM
  (
    SELECT
      SUM(b.amount) AS Revenue
    FROM Appointment a
    JOIN Billing b
      ON a.patient_id=b.patient_id
    GROUP BY doctor_id
  ) Y
);

```

doctor_id	first_name	last_name	AppointmentCount	Revenue
-----------	------------	-----------	------------------	---------

EXPLANATION: The query calculates the number of appointments and total revenue for each doctor. Two separate subqueries are then used inside the **HAVING** clause:

- One calculates the average number of appointments.

- The other calculates the average revenue.

The doctor's performance metrics are compared against both averages, and only doctors exceeding both benchmarks are returned. Although the query produces the correct result, similar calculations are repeated multiple times, making it longer and harder to maintain.

VERSION B: CTE

--Version B: CTE

WITH DoctorStats AS

```
(
  SELECT
    d.doctor_id,
    d.first_name,
    d.last_name,

    COUNT(a.appt_id) AS AppointmentCount,
    SUM(b.amount) AS Revenue

  FROM Doctor d

  JOIN Appointment a
    ON d.doctor_id = a.doctor_id

  JOIN Billing b
    ON a.patient_id = b.patient_id

  GROUP BY
    d.doctor_id,
    d.first_name,
    d.last_name
)
```

```
SELECT *
FROM DoctorStats
WHERE AppointmentCount >
(
  SELECT AVG(AppointmentCount)
  FROM DoctorStats
)
AND Revenue >
(
  SELECT AVG(Revenue)
  FROM DoctorStats
);
```

doctor_id	first_name	last_name	AppointmentCount	Revenue
-----------	------------	-----------	------------------	---------

EXPLANATION: The CTE named **DoctorStats** calculates all important statistics for each doctor in a single step, including appointment count and revenue generated. The main query

then references this CTE to compare doctors against the average values. Since the same dataset is reused multiple times, the query becomes cleaner and easier to understand.

Which is Better?

The CTE version is preferable because it avoids recalculating the same information repeatedly. When a query requires multiple references to the same dataset, a CTE significantly improves maintainability and readability.

3. COMPLEX MULTISTEP ANALYSIS

VERSION A: Nested Subqueries

--Version A: Nested Subqueries

```
SELECT
  DeptRevenue.dept_name,
  DeptRevenue.AvgDeptSpending

FROM
(
  SELECT
    dp.dept_name,
    AVG(b.amount) AS AvgDeptSpending

  FROM Department dp

  JOIN Doctor d
    ON dp.dept_id=d.dept_id

  JOIN Appointment a
    ON d.doctor_id=a.doctor_id

  JOIN Billing b
    ON a.patient_id=b.patient_id

  GROUP BY dp.dept_name

) DeptRevenue

WHERE AvgDeptSpending >

(
  SELECT AVG(amount) * 1.20
  FROM Billing
);
```

	dept_name	AvgDeptSpending
1	Emergency	14616.666666

EXPLANATION: The query first calculates the average spending for each department by joining departments, doctors, appointments, and billing records. This result is treated as a derived table. A second subquery calculates the overall hospital average billing amount. Finally, the outer query compares each department's average spending against the hospital average increased by 20%. The logic is correct, but it becomes difficult to read because multiple levels of nested queries are involved.

VERSION B: Multiple CTEs

--Version B: Multiple CTEs

```
WITH HospitalAverage AS
(
    SELECT
        AVG(amount) AS AvgHospitalBill
    FROM Billing
),
DepartmentAverage AS
(
    SELECT
        dp.dept_name,
        AVG(b.amount) AS AvgDeptBill

    FROM Department dp

    JOIN Doctor d
        ON dp.dept_id = d.dept_id

    JOIN Appointment a
        ON d.doctor_id = a.doctor_id

    JOIN Billing b
        ON a.patient_id = b.patient_id

    GROUP BY dp.dept_name
)

SELECT
    da.dept_name,
    da.AvgDeptBill
FROM DepartmentAverage da
CROSS JOIN HospitalAverage ha
WHERE da.AvgDeptBill > ha.AvgHospitalBill * 1.20;
```

	dept_name	AvgDeptSpending
1	Emergency	14616.666666

EXPLANATION: The first CTE, **HospitalAverage**, calculates the average billing amount across the entire hospital. The second CTE, **DepartmentAverage**, calculates the average billing amount for each department. The final query combines these two datasets and compares department performance against the hospital benchmark. By separating the calculations into logical steps, the overall query becomes much easier to understand and troubleshoot.

Which is Better?

The multiple CTE approach is the better solution because it breaks a complex problem into smaller, meaningful components. This improves readability, debugging, and future maintenance.

Scenario	Better Choice	Reason
Simple Filtering	CTE	Easier to read
Multiple References	CTE	Reuse results without recalculating
Complex Multi-Step	Multiple CTEs	Breaks large logic into smaller steps
Performance	Depends on execution plan	SQL Server may optimize both similarly
Maintainability	CTE	Easier debugging and modifications

TASK 10: COMPLEX ANALYTICS CHALLENGE

1. DOCTOR PERFORMANCE SCORECARD

WITH DoctorMetrics AS

```
(  
  SELECT  
    d.doctor_id,  
    d.first_name + ' ' + d.last_name AS Doctor_Name,  
    dp.dept_name,  
  
    COUNT(  
      CASE  
        WHEN a.status = 'Completed'  
        THEN a.appt_id  
      END  
    ) AS Completed_Appointments,  
  
    SUM(ISNULL(b.amount,0)) AS Total_Revenue,  
  
    AVG(ISNULL(b.amount,0) * 1.0) AS Avg_Revenue_Per_Appointment  
  
  FROM Doctor d  
  
  JOIN Department dp  
    ON d.dept_id = dp.dept_id  
  
  LEFT JOIN Appointment a  
    ON d.doctor_id = a.doctor_id  
  
  LEFT JOIN Billing b  
    ON a.patient_id = b.patient_id  
  
  GROUP BY  
    d.doctor_id,  
    d.first_name,  
    d.last_name,  
    dp.dept_name  
) ,
```

DoctorGrowth AS

```
(  
  SELECT  
    doctor_id,  
    Doctor_Name,  
    dept_name,  
    Completed_Appointments,  
    Total_Revenue,  
    Avg_Revenue_Per_Appointment,  
  
    RANK() OVER  
    (  
      ORDER BY Total_Revenue DESC
```

```

) AS Revenue_Rank,

NTILE(4) OVER
(
    ORDER BY Total_Revenue DESC
) AS Revenue_Quartile

FROM DoctorMetrics
)

SELECT *
FROM DoctorGrowth
ORDER BY Revenue_Rank;

```

	doctor_id	Doctor_Name	dept_name	Completed_Appointments	Total_Revenue	Avg_Revenue_Per_Appointment	Revenue_Rank	Revenue_Quartile
1	9	Suresh Iyer	Emergency	1	73000.00	24333.333333	1	1
2	7	Vikas Joshi	Orthopedics	1	45200.00	15066.666666	2	1
3	4	Priya Singh	Neurology	2	29000.00	9666.666666	3	1
4	2	Neha Patel	Cardiology	2	19200.00	6400.000000	4	2
5	6	Karan Mehta	Pediatrics	1	17700.00	5900.000000	5	2
6	10	Pooja Reddy	Emergency	1	14700.00	4900.000000	6	2
7	5	Anjali Gupta	Pediatrics	2	12000.00	4000.000000	7	3
8	1	Amit Sharma	Cardiology	2	10000.00	3333.333333	8	3
9	3	Raj Verma	Neurology	2	8800.00	2933.333333	9	4
10	8	Ritu Nair	Orthopedics	1	5600.00	1866.666666	10	4

INSIGHTS DISCOVERED:

1. Doctors with higher completed appointment counts tend to generate more revenue.
2. Revenue ranking helps identify top-performing doctors.
3. Quartile grouping allows management to classify doctors as top performers, average performers, or low performers.
4. The dashboard can be used for incentive planning and performance reviews.

2. PATIENT JOURNEY ANALYSIS

```

WITH PatientVisits AS
(
    SELECT
        p.patient_id,
        p.first_name,
        p.last_name,

        MIN(a.appointment_date) AS First_Visit,
        MAX(a.appointment_date) AS Latest_Visit,

        COUNT(a.appt_id) AS Total_Visits

    FROM Patient p

```

```
LEFT JOIN Appointment a
  ON p.patient_id = a.patient_id
```

```
GROUP BY
  p.patient_id,
  p.first_name,
  p.last_name
```

```
),
```

```
PatientSpending AS
```

```
(
  SELECT
    patient_id,
    SUM(amount) AS Total_Spending
  FROM Billing
  GROUP BY patient_id
),
```

```
DiagnosisFrequency AS
```

```
(
  SELECT
    patient_id,
    diagnosis,

    ROW_NUMBER() OVER
    (
      PARTITION BY patient_id
      ORDER BY COUNT(*) DESC
    ) AS rn

```

```
FROM Medical_Record
```

```
GROUP BY
  patient_id,
  diagnosis
)
```

```
SELECT
```

```
pv.patient_id,
pv.first_name,
pv.last_name,
```

```
pv.First_Visit,
pv.Latest_Visit,
pv.Total_Visits,
```

```
ps.Total_Spending,
```

```
CASE
```

```
  WHEN pv.Total_Visits >= 3
       OR ps.Total_Spending >= 15000
  THEN 'High Risk'
```

```
  WHEN pv.Total_Visits = 2
```

```

        THEN 'Medium Risk'

        ELSE 'Low Risk'
    END AS Risk_Category

FROM PatientVisits pv

LEFT JOIN PatientSpending ps
    ON pv.patient_id = ps.patient_id

ORDER BY ps.Total_Spending DESC;

```

	patient_id	first_name	last_name	First_Visit	Latest_Visit	Total_Visits	Total_Spending	Risk_Category
1	19	Harsh	Chopra	2026-07-18	2026-07-18	1	50000.00	High Risk
2	17	Sanjay	Tiwari	2026-07-12	2026-07-12	1	25000.00	High Risk
3	7	Aditya	Gupta	2025-01-25	2025-03-12	2	18000.00	High Risk
4	4	Kavya	Rao	2025-01-18	2026-08-01	2	15000.00	High Risk
5	2	Sneha	Patel	2025-01-12	2026-07-25	2	12000.00	Medium Risk
6	15	Deepak	Malhotra	2025-02-15	2025-02-15	1	8000.00	Low Risk
7	20	Divya	Menon	2026-07-20	2026-07-20	1	7000.00	Low Risk
8	9	Nitin	Singh	2025-02-01	2025-03-18	2	5000.00	Medium Risk
9	13	Vivek	Reddy	2025-02-10	2025-02-10	1	4500.00	Low Risk
10	10	Meera	Kapoor	2025-02-03	2025-03-20	2	4200.00	Medium Risk
11	8	Pallavi	Joshi	2025-01-28	2025-03-15	2	3500.00	Medium Risk
12	12	Ritika	Das	2025-02-08	2025-02-08	1	3000.00	Low Risk
13	1	Rahul	Shah	2025-01-10	2026-07-22	2	2500.00	Medium Risk
14	11	Aakash	Sharma	2025-02-05	2025-02-05	1	2500.00	Low Risk
15	5	Rohan	Verma	2025-01-20	2026-08-03	2	2200.00	Medium Risk
16	14	Ananya	Iyer	2025-02-12	2025-02-12	1	2000.00	Low Risk
17	3	Arjun	Mehta	2025-01-15	2026-07-28	2	1800.00	Medium Risk
18	16	Pooja	Kulkarni	2026-07-10	2026-07-10	1	1500.00	Low Risk
19	6	Priya	Nair	2025-01-22	2025-03-10	2	1200.00	Medium Risk
20	18	Neelam	Bansal	2026-07-15	2026-07-15	1	900.00	Low Risk

INSIGHTS DISCOVERED:

1. Patients with frequent visits often generate higher healthcare costs.
2. High-risk patients can be prioritized for follow-up care.
3. First and latest visit dates reveal patient retention patterns.
4. Spending trends help identify patients requiring long-term treatment.

3. REVENUE TREND ANALYSIS

```

WITH DailyRevenue AS
(
    SELECT

```

```

        bill_date,
        SUM(amount) AS Daily_Revenue
    FROM Billing
    GROUP BY bill_date
),

```

RevenueAnalysis AS

```

(
    SELECT
        bill_date,
        Daily_Revenue,

        SUM(Daily_Revenue)
        OVER
        (
            ORDER BY bill_date
        ) AS Running_Total,

        AVG(Daily_Revenue * 1.0)
        OVER
        (
            ORDER BY bill_date
            ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
        ) AS Moving_Average
    FROM DailyRevenue
),

```

DepartmentRevenue AS

```

(
    SELECT
        dp.dept_name,
        SUM(b.amount) AS Revenue

    FROM Department dp

    JOIN Doctor d
        ON dp.dept_id = d.dept_id

    JOIN Appointment a
        ON d.doctor_id = a.doctor_id

    JOIN Billing b
        ON a.patient_id = b.patient_id

    GROUP BY dp.dept_name
)

```

```

SELECT
    bill_date,
    Daily_Revenue,
    Running_Total,
    Moving_Average
FROM RevenueAnalysis

```


ORDER BY bill_date;

	bill_date	Daily_Revenue	Running_Total	Moving_Average
1	2025-01-10	2500.00	2500.00	2500.000000
2	2025-01-12	12000.00	14500.00	7250.000000
3	2025-01-15	1800.00	16300.00	5433.333333
4	2025-01-18	15000.00	31300.00	9600.000000
5	2025-01-20	2200.00	33500.00	6333.333333
6	2025-01-22	1200.00	34700.00	6133.333333
7	2025-01-25	18000.00	52700.00	7133.333333
8	2025-01-28	3500.00	56200.00	7566.666666
9	2025-02-01	5000.00	61200.00	8833.333333
10	2025-02-03	4200.00	65400.00	4233.333333
11	2025-02-05	2500.00	67900.00	3900.000000
12	2025-02-08	3000.00	70900.00	3233.333333
13	2025-02-10	4500.00	75400.00	3333.333333
14	2025-02-12	2000.00	77400.00	3166.666666
15	2025-02-15	8000.00	85400.00	4833.333333
16	2025-03-01	1500.00	86900.00	3833.333333
17	2025-03-05	25000.00	111900.00	11500.000000
18	2025-03-08	900.00	112800.00	9133.333333
19	2025-03-12	50000.00	162800.00	25300.000000
20	2025-03-15	7000.00	169800.00	19300.000000

-- DEPARTMENT REVENUE BREAKDOWN QUERY

```
WITH DepartmentRevenue AS
(
    SELECT
        dp.dept_name,
        SUM(b.amount) AS Revenue
    FROM Department dp
    JOIN Doctor d
        ON dp.dept_id = d.dept_id
    JOIN Appointment a
        ON d.doctor_id = a.doctor_id
    JOIN Billing b
        ON a.patient_id = b.patient_id
    GROUP BY dp.dept_name
)
SELECT
```

```

dept_name,
Revenue,

ROUND
(
    Revenue * 100.0
    /
    SUM(Revenue) OVER(),
    2
) AS Revenue_Percentage

FROM DepartmentRevenue

ORDER BY Revenue DESC;

```

	dept_name	Revenue	Revenue_Percentage
1	Emergency	87700.00	37.290000
2	Orthopedics	50800.00	21.600000
3	Neurology	37800.00	16.070000
4	Pediatrics	29700.00	12.630000
5	Cardiology	29200.00	12.410000

INSIGHTS DISCOVERED:

1. Running totals show cumulative hospital growth over time.
2. Moving averages reveal underlying revenue trends and reduce the impact of one-time spikes.
3. Department revenue percentages identify which departments contribute most to overall hospital earnings.
4. Revenue concentration can help management allocate resources more effectively.
5. Departments generating the highest share of revenue may require additional staffing and infrastructure support.